# High-Speed Optimization with Backtracking Line Search

Motupalli Hemanth Kumar

**Abstract**

This report details the implementation and benchmarking of custom numerical optimization algorithms designed for scalable machine learning models. The study evaluates the integration of Backtracking Line Search and Hessian scaling into Gradient Descent, Newton's Method, and the Quasi-Newton BFGS algorithm to optimize convergence rates on complex, high-dimensional functions.

## 1 Hessian Scaling in Gradient Descent

The standard Gradient Descent algorithm often struggles with ill-conditioned objective functions. We analyzed the quadratic function $f(x) = x_1^2 + 4x_1x_2 + 1600x_2^2$.

The theoretical Hessian matrix is $H = \begin{bmatrix} 2 & 4 \\ 4 & 3200 \end{bmatrix}$, yielding a condition number of approximately 1604, indicating a highly ill-conditioned landscape.

To accelerate convergence, we applied a diagonal scaling matrix $D_k$ derived from the inverse of the Hessian's diagonal elements. **Results:** Using a tolerance of $10^{-12}$ and $\rho = 0.5$, standard Gradient Descent reached the maximum limit of 10,000 iterations without converging. Conversely, the Scaled Gradient Descent converged to the optimal minimum in merely **17 iterations**.

## 2 Newton's Method and Line Search

We evaluated Newton's Method on the strictly convex function $q(x) = \sqrt{x_1^2 + 4} + \sqrt{x_2^2 + 4}$, starting at $x_0 = (2, 2)$ with a tolerance of $10^{-9}$.

The update rule is given by:

$$x_{k+1} = x_k - \eta_k(\nabla^2 f(x_k))^{-1}\nabla f(x_k)$$

**Results:** When utilizing a fixed step length ($\eta_k = 1$), the algorithm oscillated severely and failed to converge within 100 iterations. However, implementing an inexact Backtracking Line Search to dynamically determine the optimal step size allowed the algorithm to converge perfectly to the global minimum $(0, 0)$ in exactly **1 iteration**.

## 3 Quasi-Newton BFGS Scalability

The Broyden-Fletcher-Goldfarb-Shanno (BFGS) algorithm avoids explicit computation of the inverse Hessian matrix, making it suitable for high-dimensional problems. We

benchmarked the algorithm on the generalized Rosenbrock function for $n$ variables:

$$f(x) = \sum_{i=1}^{n-1} [4(x_i^2 - x_{i+1})^2 + (x_i - 1)^2]$$

**Benchmarking Results:** The BFGS implementation successfully minimized the complex, non-convex landscape across various dimensionalities, demonstrating high scalability.

- **N = 1000:** Converged in 97 iterations (Approx. 2.09 seconds)
- **N = 2500:** Converged in 93 iterations (Approx. 10.70 seconds)
- **N = 5000:** Converged in 113 iterations (Approx. 44.83 seconds)

# 4 Conclusion

The integration of Backtracking Line Search and targeted scaling matrices drastically accelerates convergence rates. The custom BFGS implementation efficiently resolves large-scale regularized problems, proving highly effective for dynamic machine learning applications.